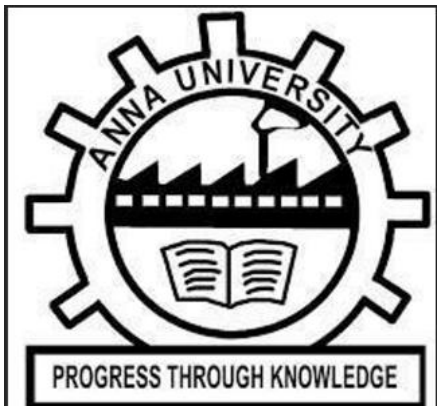


COLLEGE OF ENGINEERING, GUINDY,

ANNA UNIVERSITY

CHENNAI 600025

EC23E34 - Design and Verification using System Verilog

DESIGN AND VERIFICATION OF AN APB-COMPLIANT RADIX-4 BOOTH MULTIPLIER

Mini Project Report

Submitted By

PRIYADHARSHAN L – 2023105055

SAABIQ U A – 2023105514

KRISHNASHREE D – 2023105539

In partial fulfilment for the award of the degree of BACHELOR OF ENGINEERING

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

Algorithm for Design Code

Radix-4 Booth Multiplier (Booth_CLA_Top)

Input: X, Y (signed 8-bit). **Output:** Product[15:0], Overflow.

Step 1: Take X as the multiplier and Y as the multiplicand.

Step 2: Form the 10-bit value X_ext = {X[7], X, 0}.

Step 3: For i = 0 to 3, read the triplet X_ext[2i+2 : 2i] and map it to a digit. Store the 3-bit code of each digit in op.

Step 4: Sign-extend Y to 16 bits. For each group i, select 0, Y, 2Y, -Y or -2Y according to the code, and shift it left by 2i bits to get PP0 to PP3.

Step 5: CSA stage 1: add PP0, PP1 and PP2 to get a sum S1 (bitwise XOR) and a carry C1 (bitwise majority).

Step 6: CSA stage 2: add S1, C1 shifted left by one, and PP3 to get S2 and C2.

Step 7: Final adder: add S2 and C2 shifted left by one with the 16-bit carry-look-ahead adder (four 4-bit blocks). The result is Product The adder's carry-out is available as a separate output signal.

Step 8: Stop.

APB Master Controller (apb_master_controller)

Input: req_valid, req_write, req_addr, req_wdata, PRDATA, PREADY, PSLVERR. **Output:** APB bus signals, req_ready, rsp_valid, rsp_rdata, rsp_error.

Step 1: On reset, set state = IDLE and clear all registers.

Step 2: IDLE: set req_ready = 1. If req_valid = 1, store req_addr, req_wdata and req_write, clear rsp_error and go to SETUP; otherwise stay in IDLE.

Step 3: SETUP: set PSEL = 1 and PENABLE = 0, drive PADDR, PWDATA and PWRITE from the stored request, and go to ACCESS.

Step 4: ACCESS: set PSEL = 1 and PENABLE = 1. If PREADY = 1, store PRDATA in rsp_rdata and PSLVERR in rsp_error and go to RESP; otherwise remain in ACCESS (wait state).

Step 5: RESP: set rsp_valid = 1 and go to IDLE.

APB Slave (apb_radix4_slave)

Input: PADDR, PSEL, PENABLE, PWRITE, PWDATA. **Output:** PRDATA, PREADY, PSLVERR.

Step 1: On reset, clear operand_a, operand_b, result_reg and done_reg.

Step 2: Set PREADY = 1 permanently. Define access = PSEL and PENABLE.

Step 3: Compare PADDR with 0x00, 0x04, 0x08, 0x0C and 0x10; the address is valid if it matches one of them.

Step 4: If access = 1 and the address is invalid, set PSLVERR = 1 and make no register update.

Step 5: Write (access, PWRITE = 1, valid address): 0x00 stores PWDATA[7:0] in operand_a; 0x04 stores PWDATA[7:0] in operand_b; 0x08 with PWDATA[0] = 1 copies the multiplier output to result_reg and sets done_reg. Writes to 0x0C and 0x10 are ignored.

Design and Verification of an APB-Compliant Radix-4 Booth Multiplier

1. Aim

To design an APB-compliant peripheral around an existing signed 8-bit Radix-4 Booth multiplier, and to verify it with a class-based SystemVerilog testbench (driver, monitor, scoreboard, protocol assertions and functional coverage) simulated in Synopsys VCS.

1.1 Introduction

The Advanced Peripheral Bus (APB) is the AMBA interface used to connect low-bandwidth peripherals to a processor subsystem. In this project the multiplier is wrapped in an APB slave so that it can be programmed through memory-mapped registers: operands are written, a START command is issued, and the product and status are read back over the bus.

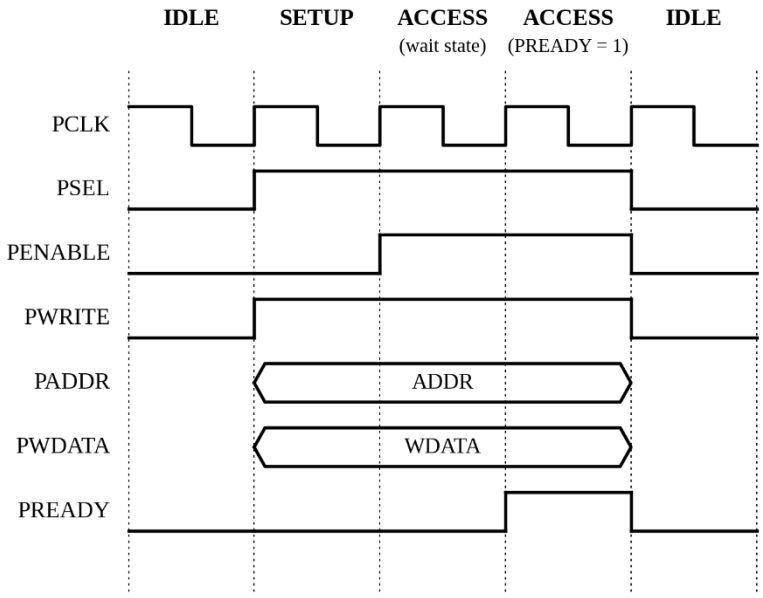
The design has three parts: an APB master controller, an APB slave with five registers, and the multiplier (Booth encoder, partial-product generator, two carry-save adder stages and a 16-bit carry-look-ahead adder). The verification environment checks the arithmetic result, register access, APB protocol behaviour, wait-state handling and invalid-address handling. Simulation was run with Synopsis VCS X-2025.06-SP1_Full64.

1.2 APB Protocol Overview

APB is a simple, non-pipelined synchronous bus. A master starts a transfer and the selected slave completes it before the next transfer begins. Every transfer passes through the following phases:

- **IDLE:** PSEL = 0 and PENABLE = 0; no transfer in progress.
- **SETUP:** PSEL = 1 and PENABLE = 0 for one clock cycle; address, direction and write data are presented.
- **ACCESS:** PSEL = 1 and PENABLE = 1; the transfer completes at the rising edge where PREADY = 1. While PREADY = 0 the slave is inserting wait states and the master must hold PADDR, PWRITE and PWDATA unchanged.

The below table lists the signals used in this design and shows a write transfer with one wait state.



Step 6: Read (PSEL = 1, PWRITE = 0): return operand_a or operand_b sign-extended (0x00, 0x04), result_reg sign-extended (0x0C), {31'b0, done_reg} (0x10), and 0 for 0x08 or any other address.

Step 7: The multiplier continuously computes the product of operand_a and operand_b.

Top Level (apb_radix4_top)

Step 1: Instantiate apb_master_controller and apb_radix4_slave.

Step 2: Connect PADDR, PSEL, PENABLE, PWRITE and PWDATA from the master to the slave, and PRDATA, PREADY and PSLVERR from the slave to the master.

Step 3: Copy all eight bus signals to the mon_* outputs for the testbench observers.

Algorithm for Testbench Code

Top-Level Test Sequence (tb_top)

Step 1: Generate a 100 MHz clock (10 ns period); instantiate the DUT, the interface connections, the assertion instances and the coverage module.

Step 2: Hold PRESETn low for 4 clock cycles, then release it.

Step 3: Create the driver, monitor and scoreboard; start the monitor in the background.

Step 4: Execute the directed multiplication test cases using the test_multiplication() task. The task accepts signed 8-bit operands A and B and calculates the expected product using signed arithmetic.

Step 5: Write operand A to address 0x00 and operand B to address 0x04, sign-extending each operand to 32 bits. Use the driver to perform the transfers and terminate the simulation with $fatal if either transfer reports an error.

Step 6: Write 1 to CONTROL (0x08); the scoreboard computes the expected product, -91.

Step 8: Read the RESULT register, compare the output with the expected product, and update pass/fail counters. Repeat for all 16 directed operand pairs.

Step 9: Test invalid address 0x14 and verify PSLVERR and rsp_error**.**

Step 10: Report scoreboard results and terminate on failures.

Step 11: Display functional coverage counters.

Step 12: Run the independent wait-state assertion test with two wait cycles.

Step 13: Use a timeout guard at 10000 time units and call $finish after verification and reporting.

.Driver (apb_driver.transfer)

Step 1: At a falling clock edge, wait until req_ready = 1 (master in IDLE).

Step 2: Drive req_write, req_addr and req_wdata, and set req_valid = 1.

Step 3: Wait for the rising edge at which req_ready = 1; the request is accepted.

Step 4: At the next falling edge, clear req_valid.

Step 5: Wait for the rising edge at which rsp_valid = 1.

Step 6: Return rsp_rdata as read_data and rsp_error as error.

APB signals used in the design

Signal	Width	Driven by	Function
PCLK	1	System	Clock; all transfers are sampled on the rising edge
PRESETn	1	System	Active-low reset
PADDR	32	Master	Register address
PSEL	1	Master	Slave select; high in SETUP and ACCESS
PENABLE	1	Master	High only in the ACCESS phase
PWRITE	1	Master	1 = write transfer, 0 = read transfer
PWDATA	32	Master	Write data
PRDATA	32	Slave	Read data
PREADY	1	Slave	Low extends ACCESS (wait state); high completes the transfer
PSLVERR	1	Slave	High signals that the transfer failed

1.3 Peripheral Specification

The peripheral uses a 32-bit data bus and exposes five registers. The operands are signed 8-bit values taken from bits [7:0] of the write data.

Register map

Offset	Register	Access	Description
0x00	OPERAND_A	Read/Write	Signed 8-bit operand X (the Booth-encoded multiplier). Read back sign-extended to 32 bits.
0x04	OPERAND_B	Read/Write	Signed 8-bit operand Y (the multiplicand). Read back sign-extended to 32 bits.
0x08	CONTROL	Write	Bit 0 = START. Reads return 0.
0x0C	RESULT	Read only	16-bit signed product, sign-extended to 32 bits.
0x10	STATUS	Read only	Bit 0 = DONE.

Monitor (apb_monitor.run)

Step 1: At every rising clock edge, check whether PRESETn, PSEL, PENABLE and PREADY are all 1.

Step 2: If so and PWRITE = 1, print WRITE with address, PWDATA and PSLVERR.

Step 3: If so and PWRITE = 0, print READ with address, PRDATA and PSLVERR.

Step 4: Repeat forever without driving any signal.

Scoreboard (apb_scoreboard)

Step 1: process_write(addr, data): for address 0x00 store data[7:0] in model_a; for 0x04 store data[7:0] in model_b.

Step 2: For address 0x08 with data[0] = 1, compute expected_product = model_a x model_b as a signed 16-bit value.

Step 3: check_result(observed): sign-extend expected_product to 32 bits.

Step 4: If observed equals the extended value (using ===), increase passed; otherwise increase failed and report an error.

Step 5: report(): print the passed and failed counts.

Assertions (apb_assertions)

Step 1: Set past_valid after the first clock edge following reset; disable all checks while PRESETn = 0.

Step 2: setup_before_access: when PSEL and PENABLE are high and PENABLE was low in the previous cycle, the previous cycle must have had PSEL = 1 and PENABLE = 0.

Step 3: stable_during_wait: when PSEL = 1, PENABLE = 1 and PREADY = 0, the next cycle must keep PSEL and PENABLE high and keep PADDR, PWRITE and PWDATA unchanged.

Step 4: access_requires_select: whenever PENABLE = 1, PSEL must be 1.

Step 5: Report $error if any property fails.

Step 6: Wait-state test: raise PSEL (SETUP), raise PENABLE (ACCESS) with PREADY low for two cycles, then set PREADY = 1 and release all signals.

Functional Coverage (apb_coverage)

Step 1: Clear all counters on reset.

Step 2: At each rising edge where PSEL, PENABLE and PREADY are all 1, a transfer has completed.

Step 3: For a write, increase write_count and the counter for the address (A, B or START); for A also classify the signed value as positive, negative or zero.

Step 4: For a read, increase read_count and the counter for RESULT or STATUS.

Step 5: If the address is not one of the five registers, increase invalid_accesses.

Step 6: At the end of the test, print all counters.

Behavioural rules of the peripheral:

- On reset, the operand registers, RESULT and DONE are cleared to 0.
- Writing 1 to CONTROL bit 0 copies the multiplier output into RESULT and sets DONE.
- DONE stays set after the first START; the RTL has no clear other than reset.
- Writes to RESULT and STATUS are ignored without an error response.
- PREADY is tied to 1, so every access completes without wait states.
- An address outside the five offsets drives PSLVERR = 1 during ACCESS and changes no register.

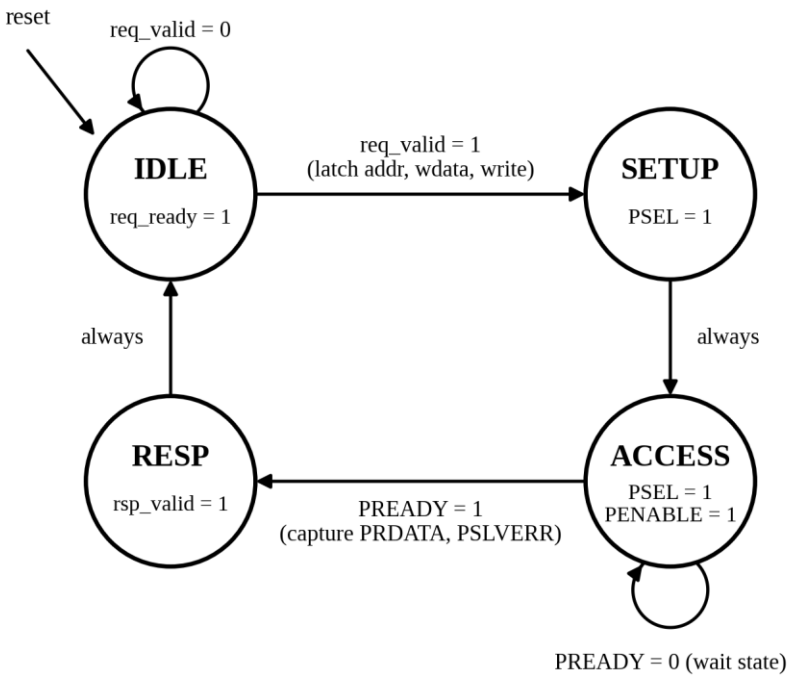
1.4 Programming Sequence

1. Write operand X to OPERAND_A (0x00).
2. Write operand Y to OPERAND_B (0x04).
3. Write 1 to CONTROL (0x08) to start the multiplication.
4. Read STATUS (0x10) and check that DONE = 1.
5. Read RESULT (0x0C) to obtain the product.

2. Block Diagram

2.1 FSM / State Diagram

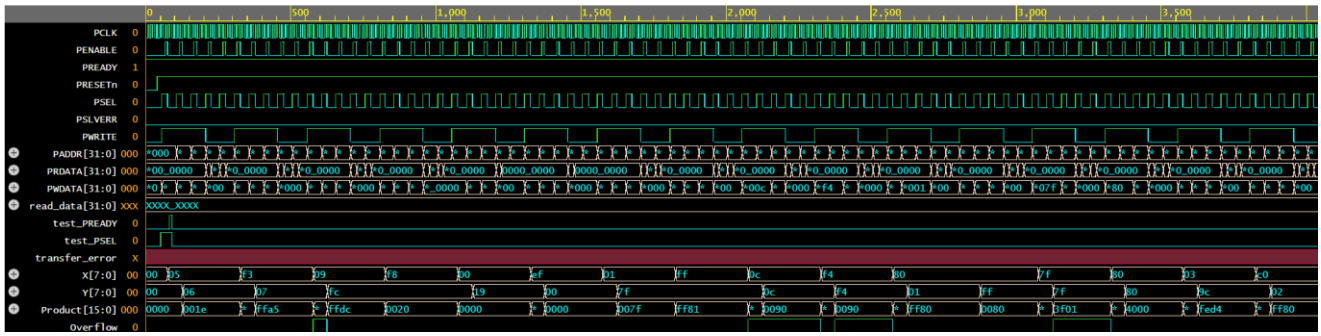
The master controller is a four-state FSM . A request is accepted only in IDLE. SETUP and ACCESS form the APB transfer, and RESP presents the captured response to the testbench for one cycle before returning to IDLE. The slave has no state machine: because PREADY is tied to 1, it responds in the ACCESS cycle using combinational address decoding and registered writes.



State diagram of the APB master controller

Results

Waveform:



Some of the test cases are mentioned here:

Test case	Operand A	Operand B	Expected product	Result
Positive × Positive	0x05 (5)	0x06 (6)	0x001E (30)	PASS
Negative × Positive	0xF3 (-13)	0x07 (7)	0xFFA5 (-91)	PASS
Positive × Negative	0x09 (9)	0xFC (-4)	0xFFDC (-36)	PASS
Negative × Negative	0xF8 (-8)	0xFC (-4)	0x0020 (32)	PASS
Equal positive operands	0x0C (12)	0x0C (12)	0x0090 (144)	PASS
Equal negative operands	0xF4 (-12)	0xF4 (-12)	0x0090 (144)	PASS
Maximum positive × maximum positive	0x7F (127)	0x7F (127)	0x3F01 (16129)	PASS
Minimum value × minimum value	0x80 (-128)	0x80 (-128)	0x4000 (16384)	PASS
Mixed-sign operands with larger magnitude	0x03 (3)	0x9C (-100)	0xFED4 (-300)	PASS
Negative operand × positive operand	0xC0 (-64)	0x02 (2)	0xFF80 (-128)	PASS

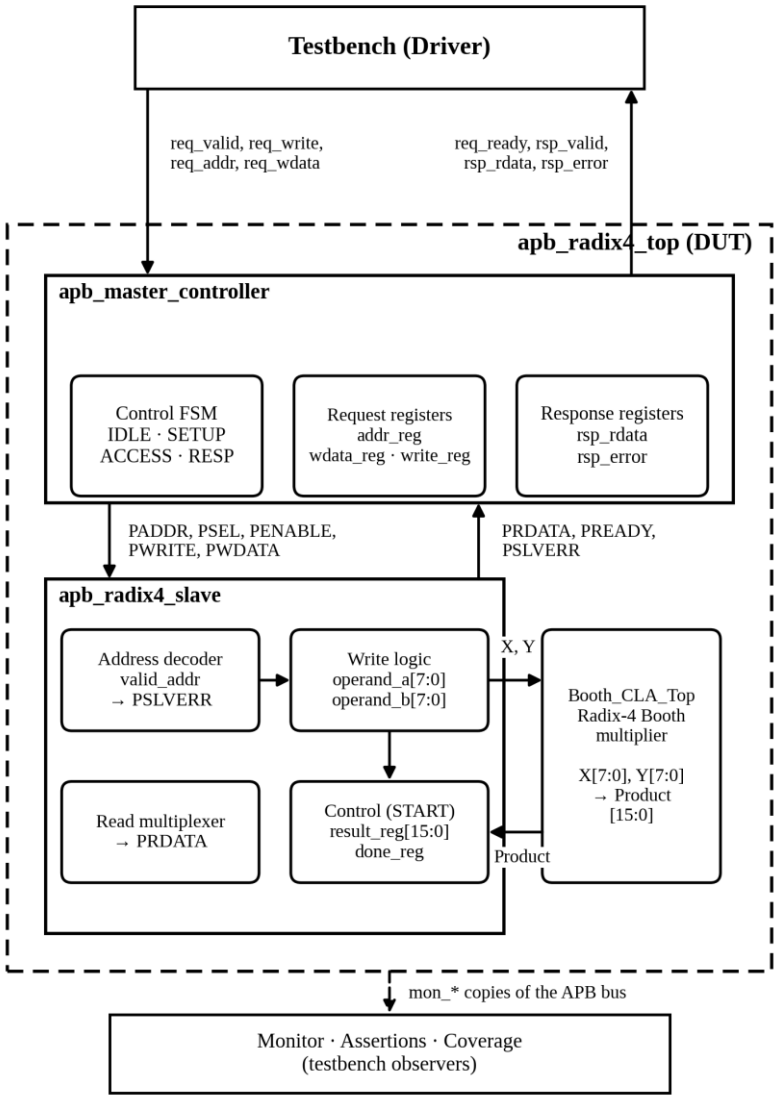
2.2 Microarchitecture

The top module apb_radix4_top connects the APB master controller and the APB slave. The testbench sends simple requests to the master (req_*) and receives responses (rsp_*); the bus signals are also copied to mon_* outputs for the testbench observers.

2.3 Block Description

- **apb_master_controller:** turns a request into an APB transfer. A four-state FSM sequences the bus, and the request and response are held in registers.
- **apb_radix4_slave:** decodes the address, stores the two operands, captures the product on START, and generates PSLVERR for unmapped addresses. A read multiplexer returns register values on PRDATA.
- **Booth_CLA_Top:** the combinational multiplier. Its inputs are the two operand registers; its 16-bit product is captured into result_reg when START is written.

A complete operation flows as follows: request, master, APB bus, operand registers, multiplier, START capture in result_reg, and finally a read of RESULT back through the read multiplexer and the master to the testbench.



Microarchitecture of the APB-based Radix-4 Booth multiplier

Scoreboard and Functional Coverage:

```
=====================================

[SCOREBOARD] Passed=16 Failed=0
ALL MULTIPLICATION TESTS PASSED

======= FUNCTIONAL COVERAGE =======
APB reads           : 32
APB writes          : 48
Operand A writes    : 16
Operand B writes    : 16
START writes        : 16
Result reads        : 16
Status reads        : 16
Invalid accesses    : 0
Positive A writes   : 6
Negative A writes   : 9
Zero A writes       : 1
==================================
$finish called from file "testbench.sv", line 639.
$finish at simulation time                 4040
```

The scoreboard ensures that all the test cases have passed.

The coverage results confirm that all 16 multiplication tests exercised the expected APB register sequence. Each test involved two operand writes, one START write, one status read, and one result read. This accounts for the 48 writes and 32 reads reported by the coverage module.

The operand A classification also confirms that positive, negative, and zero values were exercised. However, these counters measure operand A's sign distribution; they do not independently measure coverage of every operand-sign combination or every boundary-value scenario.

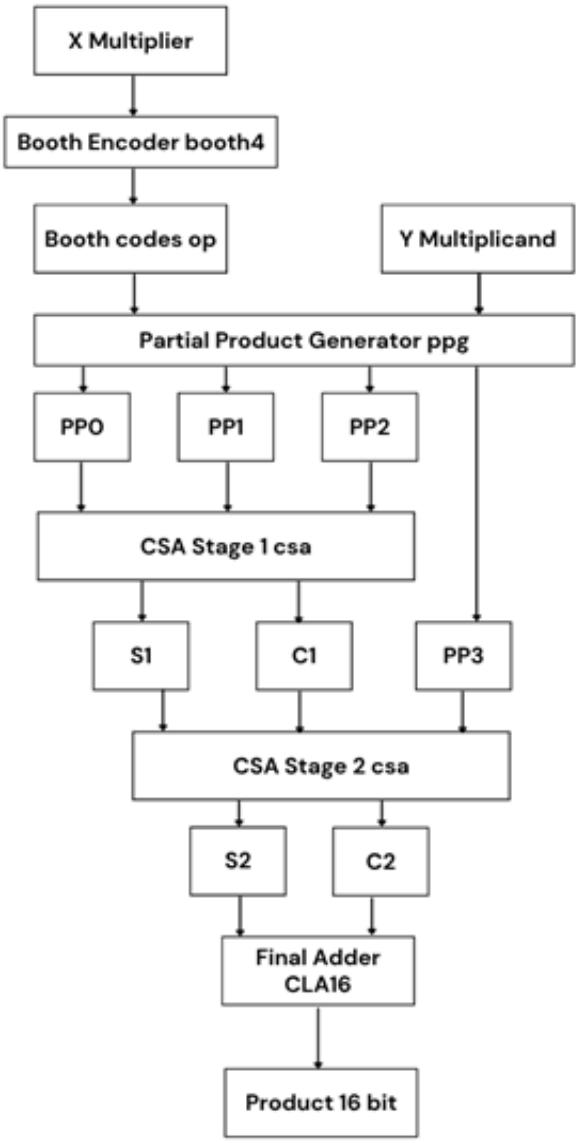
The zero count for invalid accesses is expected because this particular run focused on multiplication functionality. Invalid-address handling should be verified separately using the dedicated test that checks PSLVERR and the master's rsp_error response.

```
==================================
[MONITOR] READ  addr=00000014 data=00000000 error=1
[INVALID-ADDRESS TEST] PASS: PSLVERR=1 during ACCESS and rsp_error=1
```

The invalid-address test was executed by performing a read operation at the unmapped address 0x14. The monitor confirmed that the slave asserted PSLVERR during the ACCESS phase, and the master correctly propagated the error through rsp_error. The test passed, confirming proper error handling for invalid address accesses.

2.4 Multiplier Data Path

The flowchart shows the data path of the Radix-4 Booth multiplier. Radix-4 recoding reduces the number of partial products of an 8-bit multiplication from eight to four.



Flow chart of the Radix-4 Booth multiplier with CLA final adder

3. Design Code

Overview

The design code in design.sv has two layers. The first is the arithmetic core: an existing signed 8-bit Radix-4 Booth multiplier with a carry-look-ahead final adder. The second is the APB interface built around it, made up of a master controller, a slave with five registers, and a top-level wrapper. Together they allow operands to be written, a multiplication to be started, and the product and status to be read back through memory-mapped registers over APB.

3.1 Radix-4 Booth Multiplier

The multiplier is built from the modules **booth_4**, **ppg**, **csa** and **CLA_16**, joined in **Booth_CLA_Top**. The Booth encoder extends the multiplier X with its sign bit and an appended zero, then reads it as four overlapping 3-bit groups. Each group is converted into a digit of -2, -1, 0, +1 or +2 and stored as a 3-bit code. Groups 000 and 111 give zero, 001 and 010 give +1, 011 gives +2, 100 gives -2, and 101 and 110 give -1.

The partial-product generator sign-extends the multiplicand Y to 16 bits and, for each code, selects 0, Y, 2Y, -Y or -2Y. Each result is shifted left by two bit positions per group, which gives four partial products instead of the eight a plain 8-bit multiplication would need.

The four partial products are reduced by two carry-save adder stages. A carry-save adder is a 3:2 compressor: the sum is the bitwise XOR of the three inputs and the carry is their bitwise majority, so no carry has to travel along the word. The first stage adds the first three partial products; the second adds the first stage's sum, its carry shifted left by one, and the last partial product. A final 16-bit adder, built from four 4-bit carry-look-ahead blocks with the carry passed from block to block, adds the remaining sum and shifted carry to give the 16-bit product. The carry-out of this adder is brought out as the Overflow signal. The whole multiplier is purely combinational.

3.2 APB Master Controller

The master turns a simple request from the testbench into a full APB transfer using a four-state FSM: IDLE, SETUP, ACCESS and RESP. In IDLE the master signals that it is ready (req_ready) and waits for a request; when one arrives it stores the address, write data and direction, clears the error flag and moves to SETUP. SETUP lasts one cycle with PSEL high and PENABLE low. In ACCESS both are high, and the master stays there until PREADY goes high, at which point it registers the read data and the slave error flag and moves to RESP. In RESP the response is marked valid for one cycle, and the FSM then returns to IDLE.

The APB outputs follow directly from the state: PSEL is high in SETUP and ACCESS, PENABLE only in ACCESS, the ready signal only in IDLE and the response-valid signal only in RESP. Address, write data and direction come from the stored request, so they cannot change during a transfer. Because the master holds in ACCESS while PREADY is low, it is able to handle wait states even though this slave never inserts any.

3.3 APB Slave

The slave holds five registers: operand A at 0x00, operand B at 0x04, control at 0x08, result at 0x0C and status at 0x10. It decodes the address and ties PREADY to 1, so every access completes in the ACCESS cycle with no wait states. If the address is not one of the five offsets during ACCESS, PSLVERR is raised and no register is changed.

Writes take effect only for valid addresses during ACCESS. The two operand registers store the low 8 bits of the write data. Writing 1 to bit 0 of the control register is the START command: it copies the multiplier output into the result register and sets the DONE flag. Writes to the result and status registers are ignored without an error. Since the multiplier is combinational, its output follows the operand registers continuously, so the result register holds a meaningful value only after START has been written. DONE stays set after the first START; the only way to clear it is reset.

Reads go through a multiplexer. The operands and the result are returned sign-extended to 32 bits, the status register returns DONE in bit 0, and the control register reads as zero.

3.4 Top Level

The top-level module instantiates the master and the slave and connects the APB bus signals between them. It exposes the master's request and response ports tothe testbench, and also copies all eight bus signals to monitor outputs. This lets the monitor, assertions and coverage observe the bus without changing the design.

CODE:

booth_4.sv:

```
module booth_4 (
   input  logic signed [7:0] X,
   output logic [11:0] op
);
   int i;
   logic [9:0] X_ext;
   always_comb begin
      op = 12'b0;  // avoid latches
      // Sign extension + appended 0 (x[-1])
      X_ext = {X[7], X, 1'b0};
      for (i = 0; i < 4; i = i + 1) begin
         case (X_ext[2*i +: 3])
            3'b000, 3'b111: op[3*i +: 3] = 3'b000; // 0
            3'b001, 3'b010: op[3*i +: 3] = 3'b001; // +1
            3'b011:         op[3*i +: 3] = 3'b010; // +2
            3'b100:         op[3*i +: 3] = 3'b110; // -2
            3'b101, 3'b110: op[3*i +: 3] = 3'b111; // -1
            default:        op[3*i +: 3] = 3'b000;
         endcase
      end
   end
endmodule
```

ppg.sv:

```
module ppg (
   input  logic signed [7:0] Y,
   input  logic [11:0] op,
   output logic signed [15:0] PP0,
```

```
    output logic signed [15:0] PP1,
    output logic signed [15:0] PP2,
    output logic signed [15:0] PP3
);
    logic signed [15:0] Y_ext;
    logic signed [15:0] temp;
    always_comb begin
        // Sign extend Y
        Y_ext = {{8{Y[7]}}, Y};
        // Default values
        PP0 = 0;
        PP1 = 0;
        PP2 = 0;
        PP3 = 0;
        // Group 0
        case (op[2:0])
            3'b000: temp = 0;
            3'b001: temp = Y_ext;
            3'b010, 3'b011: temp = Y_ext << 1;  // FIXED
            3'b111: temp = -Y_ext;
            3'b110: temp = -(Y_ext << 1);
            default: temp = 0;
        endcase
        PP0 = temp;
        // Group 1
        case (op[5:3])
            3'b000: temp = 0;
            3'b001: temp = Y_ext;
            3'b010, 3'b011: temp = Y_ext << 1;
            3'b111: temp = -Y_ext;
            3'b110: temp = -(Y_ext << 1);
            default: temp = 0;
        endcase
        PP1 = temp << 2;
        // Group 2
        case (op[8:6])
            3'b000: temp = 0;
            3'b001: temp = Y_ext;
            3'b010, 3'b011: temp = Y_ext << 1;
            3'b111: temp = -Y_ext;
            3'b110: temp = -(Y_ext << 1);
            default: temp = 0;
        endcase
        PP2 = temp << 4;
        // Group 3
        case (op[11:9])
            3'b000: temp = 0;
            3'b001: temp = Y_ext;
            3'b010, 3'b011: temp = Y_ext << 1;
            3'b111: temp = -Y_ext;
            3'b110: temp = -(Y_ext << 1);
            default: temp = 0;
        endcase
        PP3 = temp << 6;
```

```
    end
endmodule
```

csa.sv:

```
module csa (
    input  logic [15:0] A, B, C,
    output logic [15:0] Sum,
    output logic [15:0] Carry
);
    assign Sum   = A ^ B ^ C;
    assign Carry = (A & B) | (B & C) | (A & C);
endmodule
```

cla.sv:

```
module CLA_16(
        input  logic [15:0] A,
        input  logic [15:0] B,
        output logic [15:0] Sum,
        output logic Cout
    );
    logic C1, C2, C3;
    Carry_look_ahead_adder CLA0 (
        .A(A[3:0]),
        .B(B[3:0]),
        .Cin(1'b0),
        .Sum(Sum[3:0]),
        .Cout(C1)
    );
    Carry_look_ahead_adder CLA1 (
        .A(A[7:4]),
        .B(B[7:4]),
        .Cin(C1),
        .Sum(Sum[7:4]),
        .Cout(C2)
    );
    Carry_look_ahead_adder CLA2 (
        .A(A[11:8]),
        .B(B[11:8]),
        .Cin(C2),
        .Sum(Sum[11:8]),
        .Cout(C3)
    );
    Carry_look_ahead_adder CLA3 (
        .A(A[15:12]),
        .B(B[15:12]),
        .Cin(C3),
        .Sum(Sum[15:12]),
        .Cout(Cout)
    );
    endmodule
```

carry_look_ahead_adder.sv:

```
module Carry_look_ahead_adder(
    input [3:0] A,
    input [3:0] B,
```

```
    input Cin,
    output [3:0] Sum,
    output Cout
);
wire [3:0] G;
wire [3:0] P;
wire C1, C2, C3;
assign P = A ^ B;
assign G = A & B;
assign C1 = G[0] | (P[0] & Cin);
assign C2 = G[1] | (P[1] & G[0]) | (P[1] & P[0] & Cin);
assign C3 = G[2] | (P[2] & G[1]) | (P[2] & P[1] & G[0])
                 | (P[2] & P[1] & P[0] & Cin);
assign Cout = G[3] | (P[3] & G[2]) | (P[3] & P[2] & G[1])
                   | (P[3] & P[2] & P[1] & G[0])
                   | (P[3] & P[2] & P[1] & P[0] & Cin);
assign Sum[0] = P[0] ^ Cin;
assign Sum[1] = P[1] ^ C1;
assign Sum[2] = P[2] ^ C2;
assign Sum[3] = P[3] ^ C3;
endmodule
```

Booth_CLA_Top.sv:

```
module Booth_CLA_Top (
    input  logic signed [7:0] X,
    input  logic signed [7:0] Y,
    output logic signed [15:0] Product,
    output logic Overflow
);
    logic [11:0] op;
    logic signed [15:0] PP0, PP1, PP2, PP3;
    logic [15:0] S1, C1;
    logic [15:0] S2, C2;
    // Booth Encoder
    booth_4 BE (.X(X), .op(op));
    // Partial Products
    ppg PPG ( .Y(Y), .op(op), .PP0(PP0), .PP1(PP1), .PP2(PP2),  .PP3(PP3) );
    // CSA Stage 1
    csa CSA1 (.A(PP0), .B(PP1), .C(PP2), .Sum(S1), .Carry(C1));
    // CSA Stage 2
    csa CSA2 (.A(S1), .B({C1[14:0],1'b0}), .C(PP3), .Sum(S2), .Carry(C2));
    // Final Adder
    CLA_16 CLA (.A(S2), .B({C2[14:0],1'b0}), .Sum(Product), .Cout(Overflow));
endmodule
```

apb_master_controller.sv:

```
module apb_master_controller (
    input  logic        PCLK,
    input  logic        PRESETn,
    // Request interface
    input  logic        req_valid,
    output logic         req_ready,
    input  logic        req_write,
    input  logic [31:0] req_addr,
```

```
    input  logic [31:0] req_wdata,
    // Response interface
    output logic        rsp_valid,
    output logic [31:0] rsp_rdata,
    output logic        rsp_error,
    // APB bus master signals
    output logic [31:0] PADDR,
    output logic        PSEL,
    output logic        PENABLE,
    output logic        PWRITE,
    output logic [31:0] PWDATA,
    input  logic [31:0] PRDATA,
    input  logic        PREADY,
    input  logic        PSLVERR
);
    typedef enum logic [1:0] {
       IDLE,
       SETUP,
       ACCESS,
       RESP
    } state_t;
    state_t state;
    logic [31:0] addr_reg;
    logic [31:0] wdata_reg;
    logic        write_reg;
    always_comb begin
       PADDR   = addr_reg;
       PWDATA  = wdata_reg;
       PWRITE  = write_reg
       PSEL    = (state == SETUP) || (state == ACCESS);
       PENABLE = (state == ACCESS);
       req_ready = (state == IDLE);
       rsp_valid = (state == RESP);
    end
    always_ff @(posedge PCLK or negedge PRESETn) begin
       if (!PRESETn) begin
          state     <= IDLE;
          addr_reg  <= '0;
          wdata_reg <= '0;
          write_reg <= 1'b0;
          rsp_rdata <= '0;
          rsp_error <= 1'b0;
       end else begin
          case (state)
             IDLE: begin
                if (req_valid) begin
                   addr_reg  <= req_addr;
                   wdata_reg <= req_wdata;
                   write_reg <= req_write;
                   rsp_error <= 1'b0;
                   state     <= SETUP;
                end
             end
```

```
            SETUP: begin
               state <= ACCESS;
            end
            ACCESS: begin
               if (PREADY) begin
                  rsp_rdata <= PRDATA;
                  rsp_error <= PSLVERR;
                  state     <= RESP;
               end
            end
            RESP: begin
               state <= IDLE;
            end

            default: begin
               state <= IDLE;
            end
         endcase
      end
   end
endmodule
```

apb_radix4_slave.sv:

```
module apb_radix4_slave (
   input  logic        PCLK,
   input  logic        PRESETn,
   input  logic [31:0] PADDR,
   input  logic        PSEL,
   input  logic        PENABLE,
   input  logic        PWRITE,
   input  logic [31:0] PWDATA,
   output logic [31:0] PRDATA,
   output logic        PREADY,
   output logic        PSLVERR
);
   localparam logic [31:0] REG_A      = 32'h00;
   localparam logic [31:0] REG_B      = 32'h04;
   localparam logic [31:0] REG_CTRL   = 32'h08;
   localparam logic [31:0] REG_RESULT = 32'h0C;
   localparam logic [31:0] REG_STATUS = 32'h10;
   logic signed [7:0] operand_a;
   logic signed [7:0] operand_b;
   logic signed [15:0] multiplier_product;
   logic              multiplier_overflow;
   logic signed [15:0] result_reg;
   logic              done_reg
   logic access;
   logic valid_addr;
   assign access = PSEL && PENABLE;
   always_comb begin
      valid_addr = 1'b1;
      case (PADDR)
         REG_A,
         REG_B,
```

```
        REG_CTRL,
        REG_RESULT,
        REG_STATUS: valid_addr = 1'b1;
        default: valid_addr = 1'b0;
    endcase
end
// Zero-wait-state APB slave.
assign PREADY  = 1'b1;
assign PSLVERR = access && !valid_addr;
// Existing multiplier: no change to its datapath.
Booth_CLA_Top u_multiplier (
    .X       (operand_a),
    .Y       (operand_b),
    .Product  (multiplier_product),
    .Overflow (multiplier_overflow)
);
// Register write logic.
always_ff @(posedge PCLK or negedge PRESETn) begin
    if (!PRESETn) begin
        operand_a <= '0;
        operand_b <= '0;
        result_reg <= '0;
        done_reg <= 1'b0;
    end else if (access && PREADY &&
                 PWRITE && valid_addr) begin
        case (PADDR)
            REG_A: begin
                operand_a <= PWDATA[7:0];
            end
            REG_B: begin
                operand_b <= PWDATA[7:0];
            end
            REG_CTRL: begin
                if (PWDATA[0]) begin
                    result_reg <= multiplier_product;
                    done_reg   <= 1'b1;
                end
            end
            default: begin
                // Writes to read-only registers are ignored.
            end
        endcase
    end
end
// Register read multiplexer.
always_comb begin
    PRDATA = '0;
    if (PSEL && !PWRITE) begin
        case (PADDR)
            REG_A:
                PRDATA = {{24{operand_a[7]}}, operand_a};
            REG_B:
                PRDATA = {{24{operand_b[7]}}, operand_b};
            REG_RESULT:
```

```
                PRDATA = {{16{result_reg[15]}}, result_reg};
            REG_STATUS:
                PRDATA = {31'b0, done_reg};
            REG_CTRL:
                PRDATA = 32'b0;
            default:
                PRDATA = 32'b0;
        endcase
    end
  end
endmodule
```

apb_radix4_top.sv:

```
module apb_radix4_top (
  input  logic        PCLK,
  input  logic        PRESETn,
  input  logic        req_valid,
  output logic        req_ready,
  input  logic        req_write,
  input  logic [31:0] req_addr,
  input  logic [31:0] req_wdata,
  output logic        rsp_valid,
  output logic [31:0] rsp_rdata,
  output logic        rsp_error,
  output logic [31:0] mon_PADDR,
  output logic        mon_PSEL,
  output logic        mon_PENABLE,
  output logic        mon_PWRITE,
  output logic [31:0] mon_PWDATA,
  output logic [31:0] mon_PRDATA,
  output logic        mon_PREADY,
  output logic        mon_PSLVERR
);
  logic [31:0] PADDR;
  logic        PSEL;
  logic        PENABLE;
  logic        PWRITE;
  logic [31:0] PWDATA;
  logic [31:0] PRDATA;
  logic        PREADY;
  logic        PSLVERR;
  assign mon_PADDR   = PADDR;
  assign mon_PSEL    = PSEL;
  assign mon_PENABLE = PENABLE;
  assign mon_PWRITE  = PWRITE;
  assign mon_PWDATA  = PWDATA;
  assign mon_PRDATA  = PRDATA;
  assign mon_PREADY  = PREADY;
  assign mon_PSLVERR = PSLVERR;
  apb_master_controller u_master (
    .PCLK      (PCLK),
    .PRESETn   (PRESETn),
    .req_valid  (req_valid),
    .req_ready  (req_ready),
```

```
        .req_write  (req_write),
        .req_addr   (req_addr),
        .req_wdata  (req_wdata),
        .rsp_valid  (rsp_valid),
        .rsp_rdata  (rsp_rdata),
        .rsp_error  (rsp_error),
        .PADDR      (PADDR),
        .PSEL       (PSEL),
        .PENABLE    (PENABLE),
        .PWRITE     (PWRITE),
        .PWDATA     (PWDATA),
        .PRDATA     (PRDATA),
        .PREADY     (PREADY),
        .PSLVERR    (PSLVERR)
    );
    apb_radix4_slave u_slave (
        .PCLK       (PCLK),
        .PRESETn    (PRESETn),
        .PADDR      (PADDR),
        .PSEL       (PSEL),
        .PENABLE    (PENABLE),
        .PWRITE     (PWRITE),
        .PWDATA     (PWDATA),
        .PRDATA     (PRDATA),
        .PREADY     (PREADY),
        .PSLVERR    (PSLVERR)
    );
endmodule
```

assertions.sv:

```
module apb_assertions (
    input logic        PCLK,
    input logic        PRESETn,
    input logic        PSEL,
    input logic        PENABLE,
    input logic        PWRITE,
    input logic [31:0] PADDR,
    input logic [31:0] PWDATA,
    input logic        PREADY
);
    logic past_valid;
    always_ff @(posedge PCLK or negedge PRESETn) begin
        if (!PRESETn)
            past_valid <= 1'b0;
        else
            past_valid <= 1'b1;
    end
    default clocking cb @(posedge PCLK); endclocking
    property setup_before_access;
        disable iff (!PRESETn)
        past_valid &&
        (PSEL && PENABLE && !$past(PENABLE))
        |-> $past(PSEL && !PENABLE);
    endproperty
```

```
    assert property (setup_before_access)
      else $error("APB assertion failed: ACCESS without SETUP");
    // Control and address must remain stable while waiting.
    property stable_during_wait;
      disable iff (!PRESETn)
      (PSEL && PENABLE && !PREADY)
      |=> (PSEL && PENABLE &&
          $stable(PADDR) &&
          $stable(PWRITE) &&
          $stable(PWDATA));
    endproperty
    assert property (stable_during_wait)
      else $error("APB controls changed during wait state");
    // PENABLE cannot be asserted while PSEL is low.
    property access_requires_select;
      disable iff (!PRESETn)
      PENABLE |-> PSEL;
    endproperty
    assert property (access_requires_select)
      else $error("PENABLE asserted without PSEL");
endmodule
```

4.Test Plan

The design is validated in layers, each checking a different aspect:

1. Compilation and elaboration of design.sv and testbench.sv in VCS.
2. A register-level sequence over APB: write both operands, write START, read STATUS and RESULT.
3. An arithmetic check: the scoreboard compares the returned product with its own expected product.
4. Protocol checking: the three assertions monitor the bus throughout the simulation.
5. A wait-state test using dedicated signals.
6. An invalid-address test that checks PSLVERR and its propagation to the master response.
7. Reporting of the functional coverage counters.

Test cases

ID	Test	Stimulus	Pass criterion
TC1	Compile and elaborate	VCS compile of design.sv and testbench.sv	No errors; hierarchy elaborates
TC2	Operand writes	Write 0xFFFFFFF3 (-13) to 0x00; write 0x00000007 to 0x04	No APB error returned by the master
TC3	START and status	Write 1 to 0x08; read 0x10	STATUS bit 0 = 1
TC4	Signed multiplication	Run 16 directed operand pairs and read result from 0x0C	DUT result matches expected product
TC5	Protocol assertions	Three assertions active on the DUT bus for the whole run	No assertion error reported
TC6	Wait state	test_PREADY low for two cycles, then high	No assertion failure
TC7	Invalid address	Read from 0x14	PSLVERR = 1 in ACCESS and rsp_error = 1
TC8	Coverage report	Counters printed at end of test	Counts reported

The status and invalid-address checks in tb_top are written as fatal checks, so a failure stops the simulation:

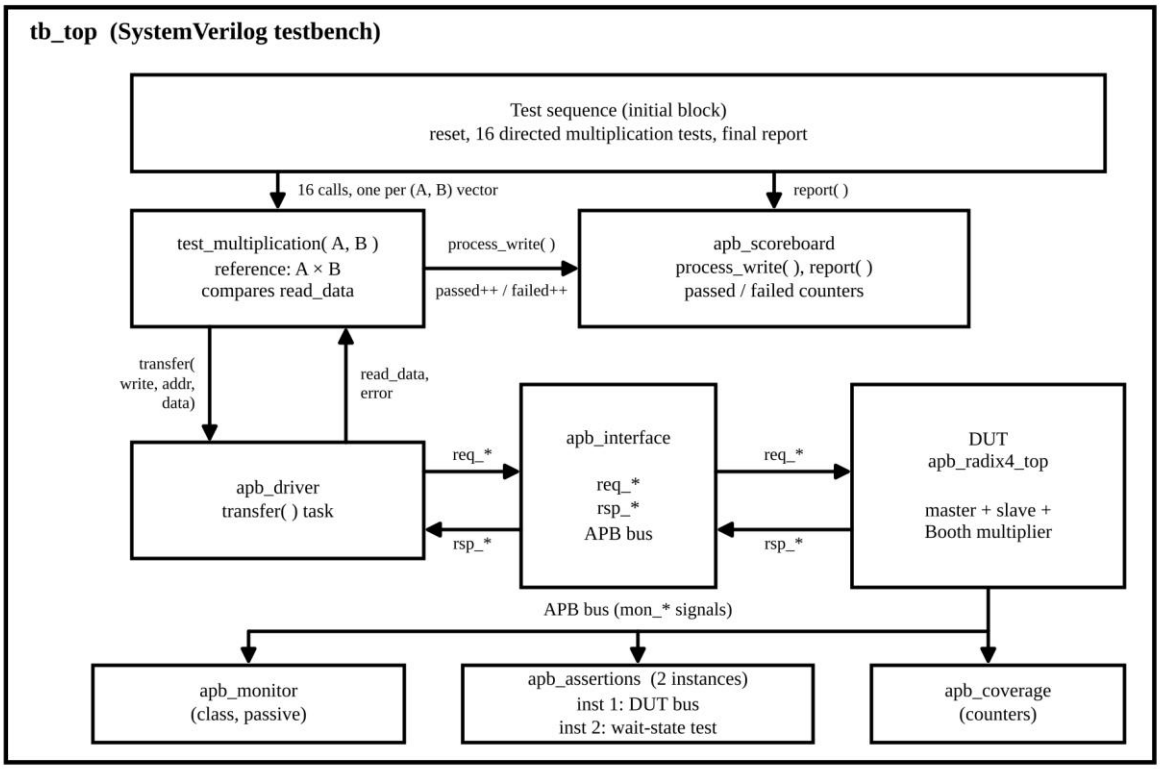
```
if (read_data[0] !== 1'b1)
   $fatal(1, "STATUS.DONE was not set");

// after reading unmapped address 0x14
if (invalid_pslverr_seen !== 1'b1)
   $fatal(1, "INVALID-ADDRESS TEST FAILED: no PSLVERR");
if (transfer_error !== 1'b1)
   $fatal(1, "INVALID-ADDRESS TEST FAILED: expected rsp_error=1");
```

The invalid_pslverr_seen flag records whether PSLVERR was asserted during the ACCESS phase for address 0x14. The test then checks transfer_error to verify that the master propagates the error through rsp_error.

5. Testbench Code

The verification environment is class-based rather than UVM. The top module tb_top generates a 100 MHz clock, instantiates the DUT, creates the driver, monitor and scoreboard objects, and runs a directed test sequence. The below figure shows how the components connect, and the table lists their roles.



SystemVerilog testbench architecture

In tb_top the DUT request/response ports are wired to the interface, and the DUT mon_* outputs are assigned to the interface bus signals. The driver and monitor therefore reach the design only through the virtual interface vif.

Testbench components

Component	Type	Role
apb_interface	interface	Groups the request/response and APB bus signals; shared with the classes as a virtual interface
apb_driver	class	Drives requests into the DUT master and collects the response
apb_monitor	class	Passively logs every completed APB transfer
apb_scoreboard	class	Reference model; computes the expected product and checks the result
apb_assertions	module	Checks APB protocol rules (defined in design.sv, instantiated in tb_top)
apb_coverage	module	Counter-based functional coverage
apb_transaction	class	Defines transaction fields, including randomizable request fields. It is not used to generate stimulus in the current directed test sequence.
tb_top	module	Clock, DUT, objects, test sequence and final reporting

5.1 Driver

The APB driver generates register access requests through the master controller using a request-response handshake. It waits until the controller is ready, submits the request, and collects the response containing the read data and error status. The driver does not directly control APB bus signals, as the master controller manages the SETUP and ACCESS phases. Requests are synchronized with the clock to ensure reliable communication.

5.2 Monitor

The APB monitor passively observes bus activity at every rising clock edge. It identifies completed transfers by checking the select, enable, and ready signals, and records the transfer direction, address, data, and error status. The monitor does not influence the design; it provides transaction information for debugging and verification.

5.3 Scoreboard

The APB scoreboard acts as an independent reference model to verify the multiplier's functionality. It maintains the operand values written to the registers and calculates the expected product when the START register is activated. When the RESULT register is read, the scoreboard compares the observed value with the expected product, including proper sign extension. Any mismatch, including unknown values, is reported as a failure.

5.4 Assertions

SystemVerilog assertions are used to verify compliance with APB protocol rules. Three properties are checked: ACCESS must follow SETUP, control signals must remain stable during wait states, and PENABLE must never be asserted without PSEL. Assertions are disabled during reset to avoid checking invalid reset conditions.

Two assertion instances are used. One monitors the actual DUT bus, while the other uses separate test signals to verify wait-state behaviour, since the DUT slave always responds with PREADY asserted. A dedicated test introduces two wait cycles to validate signal stability. Successful verification is indicated by the absence of assertion errors during legal stimulus.

5.5 Functional Coverage

Functional coverage is implemented using counters that track completed APB transactions. It records the total number of reads and writes, operand register writes, START register writes, RESULT and STATUS reads, and invalid accesses. It also classifies OPERAND_A writes as positive, negative, or zero. At the end of simulation, the coverage report displays these counts to indicate which transaction types were exercised during verification.

5.6 Transaction

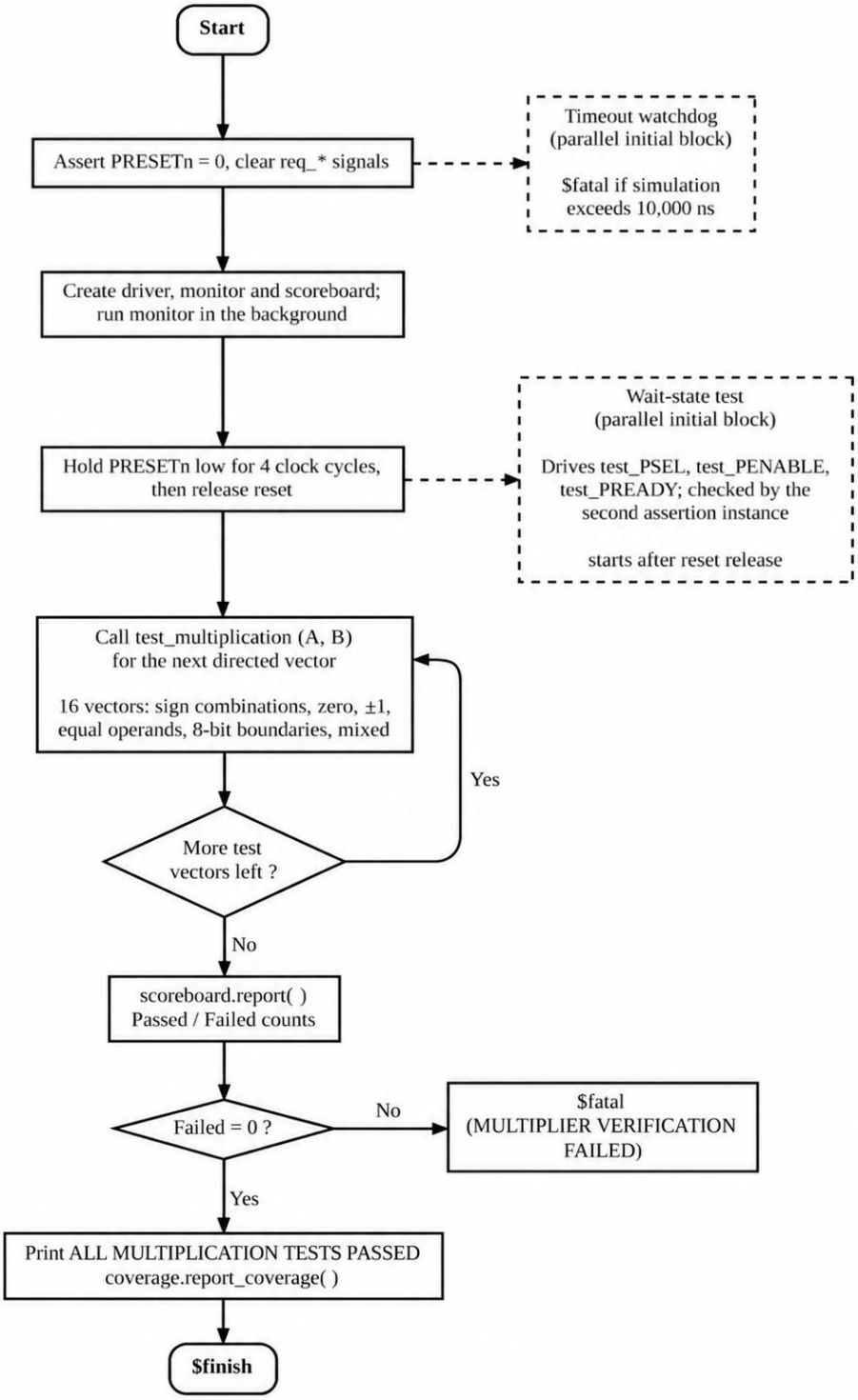
The APB transaction class defines the information associated with a single register access request. It stores fields such as the transfer direction, address, write data, read data, and error status. The transaction provides a structured representation of an APB operation, allowing the driver and other verification components to process requests and responses consistently.

5.7 Interface

The APB interface groups the signals required for communication between the master controller and the slave peripheral. It includes the APB signals, such as PCLK, PRESETn, PSEL, PENABLE, PWRITE, PADDR, PWDATA, PRDATA, PREADY, and PSLVERR, along with the master request-response handshake signals. It provides a common connection between the design and the testbench components, simplifying signal access and supporting synchronized transaction-level verification.

5.8 Top-Level Test Sequence

tb_top runs the directed sequence shown in the figure : reset, three writes, status and result reads, an invalid-address read, and the final reports. The wait-state test runs in parallel from an independent initial block.



Flow of the top-level test sequence

apb_interface.sv:

```
interface apb_interface(input logic PCLK);
   logic PRESETn;
   // Requests from testbench to synthesizable master
   logic       req_valid;
   logic       req_ready;
   logic       req_write;
   logic [31:0] req_addr;
   logic [31:0] req_wdata;
    // Responses from master
   logic       rsp_valid;
   logic [31:0] rsp_rdata;
   logic       rsp_error;
   // APB bus
   logic [31:0] PADDR;
   logic       PSEL;
   logic       PENABLE;
   logic       PWRITE;
   logic [31:0] PWDATA;
   logic [31:0] PRDATA;
   logic       PREADY;
   logic       PSLVERR;
endinterface
```

apb_driver.sv

```
class apb_driver;
   virtual apb_interface vif;
   function new(virtual apb_interface vif);
      this.vif = vif;
   endfunction
   task automatic transfer(
      input  bit          write,
      input  logic [31:0] addr,
      input  logic [31:0] data,
      output logic [31:0] read_data,
      output logic        error
   );
      // Wait until the master can accept a request.
      @(negedge vif.PCLK);
      while (!vif.req_ready)
         @(negedge vif.PCLK);
      vif.req_write = write;
      vif.req_addr  = addr;
      vif.req_wdata = data;
      vif.req_valid = 1'b1;
      // Request is accepted at a rising edge.
      do @(posedge vif.PCLK);
      while (!vif.req_ready);
      @(negedge vif.PCLK);
      vif.req_valid = 1'b0;
      // Wait for the transaction response.
      do @(posedge vif.PCLK);
      while (!vif.rsp_valid);
      read_data = vif.rsp_rdata;
```

```
        error     = vif.rsp_error;
        @(negedge vif.PCLK);
    endtask

endclass
```

apb_monitor.sv:

```
class apb_monitor;
    virtual apb_interface vif;
    function new(virtual apb_interface vif);
        this.vif = vif;
    endfunction
    task run();
        forever begin
            @(posedge vif.PCLK);
            if (vif.PRESETn &&
                vif.PSEL &&
                vif.PENABLE &&
                vif.PREADY) begin
                if (vif.PWRITE) begin
                    $display(
                        "[MONITOR] WRITE addr=%08h data=%08h error=%0b",
                        vif.PADDR, vif.PWDATA, vif.PSLVERR
                    );
                end else begin
                    $display(
                        "[MONITOR] READ  addr=%08h data=%08h error=%0b",
                        vif.PADDR, vif.PRDATA, vif.PSLVERR
                    );
                end
            end
        end
    endtask
endclass
```

apb_transaction.sv

```
class apb_transaction;
    // Transaction request
    rand bit         write;
    rand logic [31:0] addr;
    rand logic [31:0] wdata
    // Transaction response
    logic [31:0] rdata;
    logic        error;
    // Optional transaction identifier
    int unsigned transaction_id;
    function new();
        write          = 1'b0;
        addr           = '0;
        wdata          = '0;
        rdata          = '0;
        error          = 1'b0;
        transaction_id = 0;
    endfunction
```

```
    function void display(string prefix = "APB_TRANSACTION");
        $display(
            "[%s] ID=%0d OP=%s ADDR=0x%08h WDATA=0x%08h RDATA=0x%08h
ERROR=%0b",
            prefix,
            transaction_id,
            write ? "WRITE" : "READ",
            addr,
            wdata,
            rdata,
            error
        );
    endfunction
endclass
```

apb_scoreboard.sv:

```
class apb_scoreboard;
    logic signed [7:0]  model_a;
    logic signed [7:0]  model_b;
    logic signed [15:0] expected_product;
    int passed;
    int failed;
    function new();
        model_a = '0;
        model_b = '0;
        passed  = 0;
        failed  = 0;
    endfunction
    task process_write(
        input logic [31:0] addr,
        input logic [31:0] data
    );
        case (addr)
            32'h00: model_a = data[7:0];
            32'h04: model_b = data[7:0];
            32'h08: begin
                if (data[0]) begin
                    expected_product = model_a * model_b;
                    $display(
                        "[SCOREBOARD] Expected product = %0d (0x%04h)",
                        expected_product, expected_product
                    );
                end
            end
            default: ;
        endcase
    endtask
    task check_result(input logic [31:0] observed);
        logic signed [31:0] expected_extended;
        expected_extended = {
            {16{expected_product[15]}}, expected_product
        };
        if (observed === expected_extended) begin
            passed++;
```

```
            $display("[SCOREBOARD] PASS: result = %08h", observed);
        end else begin
            failed++;
            $error(
                "[SCOREBOARD] FAIL: expected %08h, observed %08h",
                expected_extended, observed
            );
        end
    endtask

    function void report();
        $display(
            "[SCOREBOARD] Passed=%0d Failed=%0d",
            passed, failed
        );
    endfunction
endclass
```

apb_coverage.sv:

```
module apb_coverage (
    input logic       PCLK,
    input logic       PRESETn,
    input logic       PSEL,
    input logic       PENABLE,
    input logic       PREADY,
    input logic       PWRITE,
    input logic [31:0] PADDR,
    input logic [31:0] PWDATA,
    input logic [31:0] PRDATA
);
    integer read_count;
    integer write_count;
    integer operand_a_writes;
    integer operand_b_writes;
    integer start_writes;
    integer result_reads;
    integer status_reads;
    integer invalid_accesses;
    integer positive_a_writes;
    integer negative_a_writes;
    integer zero_a_writes;
    always @(posedge PCLK or negedge PRESETn) begin
        if (!PRESETn) begin
            read_count         <= 0;
            write_count        <= 0;
            operand_a_writes   <= 0;
            operand_b_writes   <= 0;
            start_writes       <= 0;
            result_reads       <= 0;
            status_reads       <= 0;
            invalid_accesses   <= 0;
            positive_a_writes  <= 0;
            negative_a_writes  <= 0;
            zero_a_writes      <= 0;
```

```
    end else if (PSEL && PENABLE && PREADY) begin
      if (PWRITE) begin
        write_count <= write_count + 1
        case (PADDR)
          32'h00: begin
            operand_a_writes <= operand_a_writes + 1;
            if ($signed(PWDATA[7:0]) > 0)
              positive_a_writes <= positive_a_writes + 1;
            else if ($signed(PWDATA[7:0]) < 0)
              negative_a_writes <= negative_a_writes + 1;
            else
              zero_a_writes <= zero_a_writes + 1;
          end

          32'h04:
            operand_b_writes <= operand_b_writes + 1;
          32'h08:
            if (PWDATA[0])
              start_writes <= start_writes + 1;
          default: ;
        endcase
      end else begin
        read_count <= read_count + 1;
        case (PADDR)
          32'h0C: result_reads <= result_reads + 1;
          32'h10: status_reads <= status_reads + 1;
          default: ;
        endcase
      end
      if (PADDR != 32'h00 &&
        PADDR != 32'h04 &&
        PADDR != 32'h08 &&
        PADDR != 32'h0C &&
        PADDR != 32'h10)
        invalid_accesses <= invalid_accesses + 1;
    end
  end
  // Call this report task from tb_top at the end of simulation.
  task report_coverage;
    begin
      $display("");
      $display("======= FUNCTIONAL COVERAGE =======");
      $display("APB reads          : %0d", read_count);
      $display("APB writes         : %0d", write_count);
      $display("Operand A writes   : %0d", operand_a_writes);
      $display("Operand B writes   : %0d", operand_b_writes);
      $display("START writes       : %0d", start_writes);
      $display("Result reads       : %0d", result_reads);
      $display("Status reads       : %0d", status_reads);
      $display("Invalid accesses   : %0d", invalid_accesses);
      $display("Positive A writes  : %0d", positive_a_writes);
      $display("Negative A writes  : %0d", negative_a_writes);
      $display("Zero A writes      : %0d", zero_a_writes);
      $display("====================================");
```

```
        end
    endtask
endmodule
```

tb_top.sv:

```
module tb_top;
    logic PCLK;
    apb_interface apb_if(PCLK);
    logic [31:0] mon_PADDR;
    logic        mon_PSEL;
    logic        mon_PENABLE;
    logic        mon_PWRITE;
    logic [31:0] mon_PWDATA;
    logic [31:0] mon_PRDATA;
    logic        mon_PREADY;
    logic        mon_PSLVERR;

    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, tb_top);
    end
    // DUT response signals are included in the interface.
    apb_radix4_top dut (
        .PCLK       (PCLK),
        .PRESETn    (apb_if.PRESETn),
        .req_valid  (apb_if.req_valid),
        .req_ready  (apb_if.req_ready),
        .req_write  (apb_if.req_write),
        .req_addr   (apb_if.req_addr),
        .req_wdata  (apb_if.req_wdata)
        .rsp_valid  (apb_if.rsp_valid),
        .rsp_rdata  (apb_if.rsp_rdata),
        .rsp_error  (apb_if.rsp_error),
        .mon_PADDR   (mon_PADDR),
        .mon_PSEL    (mon_PSEL),
        .mon_PENABLE (mon_PENABLE),
        .mon_PWRITE  (mon_PWRITE),
        .mon_PWDATA  (mon_PWDATA),
        .mon_PRDATA  (mon_PRDATA),
        .mon_PREADY  (mon_PREADY),
        .mon_PSLVERR (mon_PSLVERR)
    );
    assign apb_if.PADDR   = mon_PADDR;
    assign apb_if.PSEL    = mon_PSEL;
    assign apb_if.PENABLE = mon_PENABLE;
    assign apb_if.PWRITE  = mon_PWRITE;
    assign apb_if.PWDATA  = mon_PWDATA;
    assign apb_if.PRDATA  = mon_PRDATA;
    assign apb_if.PREADY  = mon_PREADY;
    assign apb_if.PSLVERR = mon_PSLVERR;
    // Clock: 100 MHz
    initial PCLK = 1'b0;
    always #5 PCLK = ~PCLK;
    // Verification objects
```

```
apb_driver     driver;
apb_monitor    monitor;
apb_scoreboard scoreboard;
logic [31:0] read_data;
logic        transfer_error;
logic        invalid_pslverr_seen;
// Remember whether the slave asserted PSLVERR during the invalid ACCESS phase.
always_ff @(posedge PCLK or negedge apb_if.PRESETn) begin
   if (!apb_if.PRESETn)
      invalid_pslverr_seen <= 1'b0;
   else if (mon_PSEL && mon_PENABLE && mon_PREADY &&
         mon_PADDR == 32'h14 && mon_PSLVERR)
      invalid_pslverr_seen <= 1'b1;
end
// APB protocol assertions
apb_assertions assertions (
   .PCLK    (PCLK),
   .PRESETn (apb_if.PRESETn),
   .PSEL    (mon_PSEL),
   .PENABLE (mon_PENABLE),
   .PWRITE  (mon_PWRITE),
   .PADDR   (mon_PADDR),
   .PWDATA  (mon_PWDATA),
   .PREADY  (mon_PREADY)
);
// Dedicated checker for a valid wait-state test.
// These signals are independent of the real DUT bus so the DUT remains unchanged.
logic        test_PSEL;
logic        test_PENABLE;
logic        test_PWRITE;
logic [31:0] test_PADDR;
logic [31:0] test_PWDATA;
logic        test_PREADY;
apb_assertions wait_state_assertions (
   .PCLK    (PCLK),
   .PRESETn (apb_if.PRESETn),
   .PSEL    (test_PSEL),
   .PENABLE (test_PENABLE),
   .PWRITE  (test_PWRITE),
   .PADDR   (test_PADDR),
   .PWDATA  (test_PWDATA),
   .PREADY  (test_PREADY)
);
// Exercise a legal APB wait state: controls remain stable while PREADY=0.
initial begin
   test_PSEL   = 1'b0;
   test_PENABLE = 1'b0;
   test_PWRITE = 1'b0;
   test_PADDR  = 32'h00000004;
   test_PWDATA = 32'h00000007;
   test_PREADY = 1'b0;
   wait (apb_if.PRESETn === 1'b1);
   // SETUP phase: selected, PENABLE low.
   @(negedge PCLK);
```

```
    test_PSEL    = 1'b1;
    test_PENABLE = 1'b0;
    // ACCESS phase begins after a full SETUP cycle.
    @(negedge PCLK);
    test_PENABLE = 1'b1;
    test_PREADY  = 1'b0;
    // Two wait cycles. Keep PSEL/PENABLE/address/direction/data unchanged.
    repeat (2) @(negedge PCLK);
    // Complete the transfer; keep ACCESS controls stable through the next edge.
    test_PREADY = 1'b1;
    @(negedge PCLK);
    test_PSEL    = 1'b0;
    test_PENABLE = 1'b0;
    test_PREADY  = 1'b0;
    $display("[WAIT-STATE TEST] PASS: valid wait state completed without assertion failure");
end
// Functional coverage
apb_coverage coverage (
    .PCLK    (PCLK),
    .PRESETn (apb_if.PRESETn),
    .PSEL    (mon_PSEL),
    .PENABLE (mon_PENABLE),
    .PREADY  (mon_PREADY),
    .PWRITE  (mon_PWRITE),
    .PADDR   (mon_PADDR),
    .PWDATA  (mon_PWDATA),
    .PRDATA  (mon_PRDATA)
);
initial begin
    apb_if.PRESETn   = 1'b0;
    apb_if.req_valid = 1'b0;
    apb_if.req_write = 1'b0;
    apb_if.req_addr  = '0;
    apb_if.req_wdata = '0;
    driver     = new(apb_if);
    monitor    = new(apb_if);
    scoreboard = new();
    // Run passive monitor independently.
    fork
        monitor.run();
    join_none
    repeat (4) @(negedge PCLK);
    apb_if.PRESETn = 1'b1;
     // Directed multiplication tests
    // 1. Positive x Positive
    test_multiplication(8'sd5, 8'sd6);
    // 2. Negative x Positive
    test_multiplication(-8'sd13, 8'sd7);
    // 3. Positive x Negative
    test_multiplication(8'sd9, -8'sd4);
    // 4. Negative x Negative
    test_multiplication(-8'sd8, -8'sd4);
    // 5. Multiplication by zero
    test_multiplication(8'sd0, 8'sd25);
```

```
        test_multiplication(-8'sd17, 8'sd0);
        // 6. Multiplication by one and minus one
        test_multiplication(8'sd1, 8'sd127);
        test_multiplication(-8'sd1, 8'sd127);
        // 7. Equal operands
        test_multiplication(8'sd12, 8'sd12);
        test_multiplication(-8'sd12, -8'sd12);
        // 8. Boundary values for signed 8-bit operands
        test_multiplication(-8'sd128, 8'sd1);
        test_multiplication(-8'sd128, -8'sd1);
        test_multiplication(8'sd127, 8'sd127);
        test_multiplication(-8'sd128, -8'sd128);
        // 9. Mixed magnitudes
        test_multiplication(8'sd3, -8'sd100);
        test_multiplication(-8'sd64, 8'sd2);
        // Report results for the complete test sequence
        scoreboard.report();
        if (scoreboard.failed != 0)
            $fatal(1, "MULTIPLIER VERIFICATION FAILED");
        else
            $display("ALL MULTIPLICATION TESTS PASSED");
        coverage.report_coverage();
         // Access an unmapped address. The slave should assert PSLVERR,
        // which the master must return as rsp_error.
        driver.transfer(
            1'b0, 32'h14, 32'h00000000,
            read_data, transfer_error
        );
        if (invalid_pslverr_seen !== 1'b1)
            $fatal(1, "INVALID-ADDRESS TEST FAILED: PSLVERR was not asserted during
ACCESS");
        if (transfer_error !== 1'b1)
            $fatal(1, "INVALID-ADDRESS TEST FAILED: expected rsp_error=1, got %0b",
                transfer_error);
        $display("[INVALID-ADDRESS TEST] PASS: PSLVERR=1 during ACCESS and
rsp_error=1");
        $finish;
        scoreboard.report();
        if (scoreboard.failed != 0)
            $fatal(1, "TEST FAILED");
        else
            $display("TEST PASSED");
        coverage.report_coverage();
        $finish;
    end
    initial begin
        #10000;
        $fatal(1, "Simulation timeout");
    end
endmodule
```

6.Inference

The Radix-4 Booth multiplier was verified using directed test cases covering different signed operand combinations, zero values, and boundary conditions. The results were compared against the expected products to check arithmetic correctness.

The APB interface was verified using protocol assertions, register read/write operations, and an invalid-address test. The error-handling test confirmed that PSLVERR was asserted and the error was propagated to the master through rsp_error.

The driver, monitor, scoreboard, and functional coverage counters provide a structured approach to verification. Together, these components help detect design errors, monitor bus transactions, and evaluate test coverage, improving the reliability of the verification process.

7. Conclusion

This project presents the design and verification of an APB-compliant peripheral that integrates a signed 8-bit Radix-4 Booth multiplier with a 16-bit carry-look-ahead final adder. It is accessed through an APB master controller and a slave with five memory-mapped registers.

A class-based SystemVerilog testbench (driver, monitor, scoreboard, assertions, coverage counters) ran sixteen directed tests covering sign combinations, zero, ±1, equal operands, boundary values, and mixed signs. All 16 tests passed with zero failures, and coverage recorded 48 writes and 32 reads, matching the register-access sequence. Assertions check SETUP-to-ACCESS ordering, signal stability during wait states, and that PENABLE requires PSEL. A separate stimulus exercised the wait-state assertion, since the slave itself has zero wait states (PREADY tied high).

Stimulus is currently directed only. Future work includes constrained-random generation, broader operand-sign and boundary coverage, and dedicated invalid-address and error-propagation tests.